

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Reminder: Homework assignment 2

- Due: **11:59 PM** on **Friday, March 13**
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
 - Do not upload the data file.
 - Make sure to **rename your notebook file** as instructed.

Lecture 13

- Parameter Estimation:
 - Bootstrap Confidence Interval (leftover from the last class)
 - Maximum Likelihood Parameter Estimation

Previously... Bootstrapping

- Procedure
 1. Draw many resamples (with replacement) from the original dataset.
 2. Compute the statistic of interest for each resample.
 3. Examine the distribution of the computed statistics.
- Key Idea
 - The original sample serves as a stand-in for the population.
 - Repeated resampling approximates the variability we would observe from repeated sampling of the true population.

```
# Given the following
```

```
np.random.seed(22)
```

```
data = np.random.normal(50, 10, 100)  
data.mean()
```

```
49.46324599387749
```

```
# One bootstrap resample - Number of distinct values
```

```
len(set(np.random.choice(data, 100)))
```

```
65
```

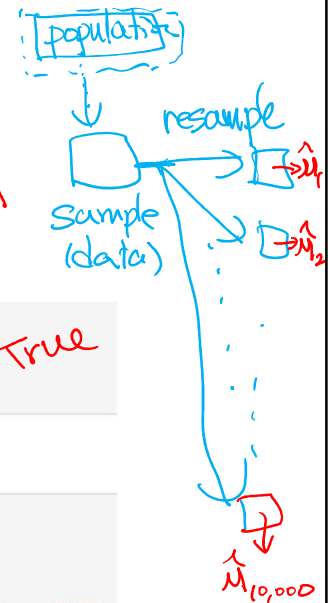
```
bootstrap_means = []
```

```
for i in range(10_000):
```

```
    returns_data_100_bootstrap = np.random.choice(data, 100)
```

```
    bootstrap_mean = returns_data_100_bootstrap.mean()
```

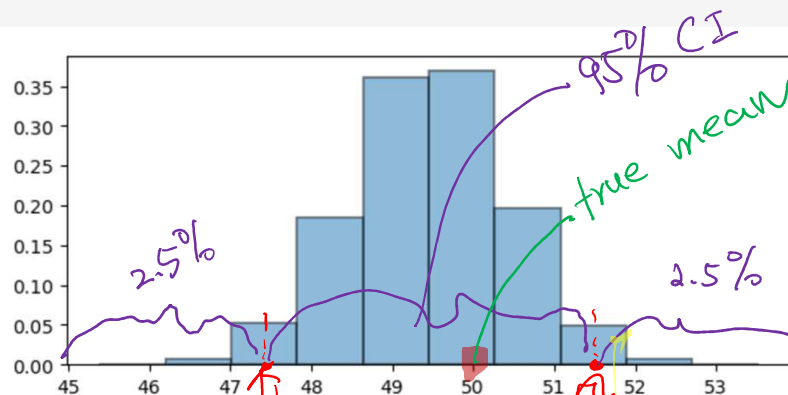
```
    bootstrap_means.append(bootstrap_mean)
```



```
plt.figure(figsize=(7, 3))
```

```
plt.hist(bootstrap_means, density=True, edgecolor='black', linewidth=1.2, alpha=0.5)
```

```
plt.show()
```



```
np.percentile(bootstrap_means, (2.5, 97.5))
```

```
array([47.49032939, 51.35670114])
```

How confident are we in the bootstrap confidence interval?

- The estimate 95% confidence interval captures the population parameter. Was that a fluke?
- To see how frequently the interval contains the parameter, we have to run the entire process over and over again.
- We will repeat the following process a number of times (say 100 times):
 - Generate a new data sample of size 100 from the population.
 - Generate 10,000 bootstrap resamples from the data sample.
 - Compute the 95% confidence interval for the mean.
- We will end up with 100 intervals, and count how many of them contain the population mean.
 - How many of these 95% confidence intervals will include our mean?

```
def compute_conf_interval(data, nb_bootstrap_iters = 10_000):  
    bootstrap_means = []  
    for i in range(nb_bootstrap_iters):  
        bootstrap_sample = np.random.choice(data, 100, replace=True)  
        bootstrap_means.append(bootstrap_sample.mean())  
    return np.percentile(bootstrap_means, (2.5, 97.5))
```

```
lower_bound = []  
upper_bound = []  
for i in range(100):  
    data = np.random.normal(50, 10, 100)  
    conf_interval = compute_conf_interval(data)  
    lower_bound.append(conf_interval[0])  
    upper_bound.append(conf_interval[1])
```

bootstrap
procedure

simulation

new sample of 100

```
conf_ints_95 = pd.DataFrame({"lower": lower_bound, "upper": upper_bound})
conf_ints_95.head()
```

	lower	upper
0	48.652341	52.444057
1	48.368890	52.473363
2	48.343475	52.276448
3	46.143426	50.403539
4	48.419776	52.061113

```
conf_ints_95.shape
```

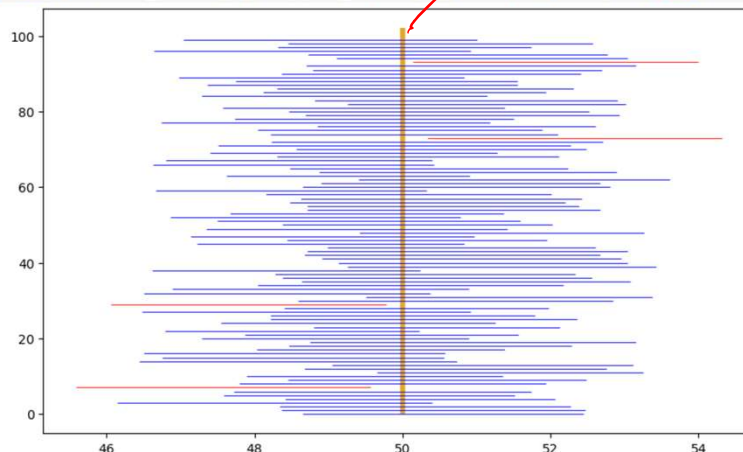
```
(100, 2)
```

```
sum((conf_ints_95["lower"] < 50) & (conf_ints_95["upper"] > 50))
```

96 ← # of 95% CIs containing the true mean.

```
plt.figure(figsize=(10, 6))
plt.vlines(50, 0, 102, color="#e2a829", linewidth=4)

for i in range(100):
    c = "blue"
    if lower_bound[i] > 50 or upper_bound[i] < 50:
        c = "red"
    plt.hlines(i, lower_bound[i], upper_bound[i], color=c, alpha=0.85, linewidth=0.75)
```



How confident are we in the bootstrap confidence interval? (cont'd.)

- If an interval doesn't cover the parameter, it's a failure.
 - There are very few of them in this case.
- Any method based on sampling has the possibility of being off.
 - If we have a 95% confidence interval, we should expect to be wrong 5% of the time.
 - Statistical expectation over a long run of experiments.
 - If you were to construct confidence intervals an extremely large number of times under the same conditions, 95% of those intervals would contain the true parameter value, leaving 5% that do not.
- The beauty of methods based on random sampling is that we can quantify how often they are likely to be off.

Misinterpretations and misunderstandings of confidence intervals

- A 95% confidence interval does not mean there is a 95% chance that a given interval contains the true parameter value.
 - The probability of a wrong interval containing the true value is 0.
- It means that 95% of such intervals constructed from repeated sampling will contain the true parameter. This distinction is subtle but fundamental to understanding statistical confidence.
- Once a specific interval has been calculated from a specific sample, it either does contain the true parameter, or it doesn't.
- The interval itself doesn't have a probability attached to it because it's fixed.

The bootstrap confidence interval

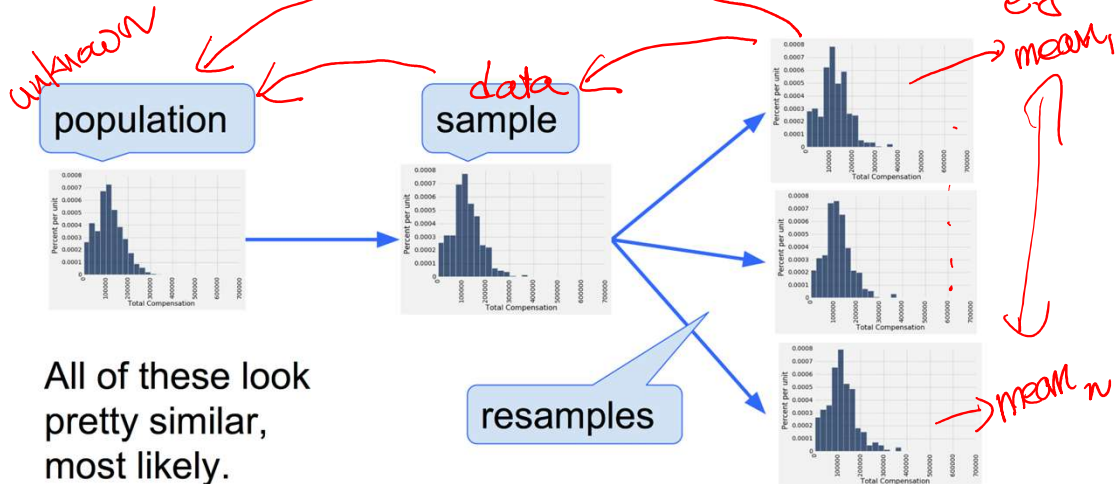
- Our interval of estimates is called a 95% confidence interval for the parameter of interest.
- 95% is called the *confidence level* of the interval.
 - In the **long run**, this *proportion of intervals* would contain the true parameter if the procedure were repeated many times.
- We say that we are *95% confident* that the process results in a good interval.

Why is the bootstrap a good idea?

$A \rightarrow B$
 $B \rightarrow C$
 $\Rightarrow A \rightarrow C.$

- Similarity by transitivity
 - The distribution of the original sample likely resembles the population.
 - “Resamples” drawn from the sample are likely to resemble the original sample.
 - So, the distributions of all the resamples are likely to resemble the population as well.
- Treat the original sample as if it were the entire population.
- Draw random resamples from the sample **with replacement**, *same size as the original*.
 - Avoids any discrepancies that may be due to size alone.
- Benefits
 - Provides a practical, data-driven estimate of sampling variability.
 - Construct confidence intervals **without strong parametric assumptions**.

Why is the bootstrap a good idea? (cont'd.)



Care in using the bootstrap

- The bootstrap is an elegant and powerful method to assess the accuracy of the estimate.
- What you need to know before using it:
 - Start with a large random sample. The larger the sample, the more reliable the bootstrapping results are likely to be.
 - To approximate the probability distribution of a statistic, it is a good idea to replicate the resampling procedure as many times as possible.
 - 10,000, in general, is a good estimate.
 - The bootstrap percentile method works well for estimating the population median or mean based on a large random sample.

When to not use the bootstrap

- The bootstrap is not useful in the following situations:
 - The goal is to estimate the minimum or maximum value in the population, or a very low or very high percentile, or parameters that are greatly influenced by rare elements of the population.
 - Lack of information in the data
 - Non-smooth behavior of the statistic: its probability distribution is non-normal, irregular, or discontinuous.
 - The original sample is very small, say less than 20 or 25.
 - Sample is not representative of the population.
 - Bootstrap samples are not representative of samples from the population.

Maximum Likelihood Parameter Estimation

Parameter estimation

- Recall the example from the previous class:

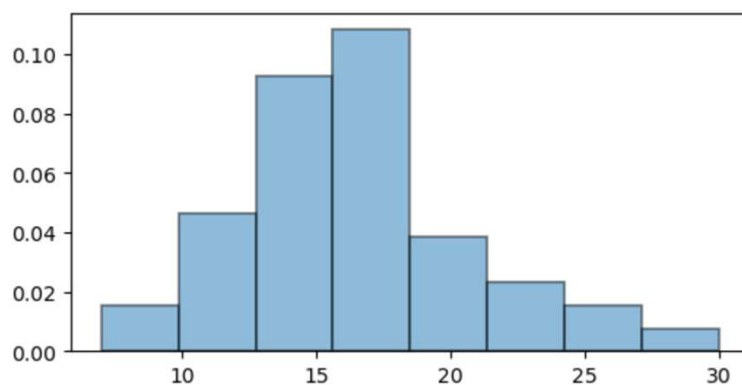
We randomly count the number of observed moving traffic citations within a 5-mile radius of UH Manoa over 90 days, spread across 12 months.

- We want to even out sampling to avoid any bias.
 - E.g., we may underestimate the number of citations during summer or overestimate during the holiday season.
- What does it mean to estimate the distribution parameter based on *maximum likelihood*?

```
citations_data = pd.read_csv("data/citations_counts.tsv", index_col="Day")
citations_data.head()
```

Counts	
Day	
0	27
1	18
2	30
3	13
4	20

```
plt.figure(figsize=(10, 6))
plt.hist(citations_data["Counts"], bins=8, density=True,
         edgecolor='black', linewidth=1.2, alpha=0.5);
```



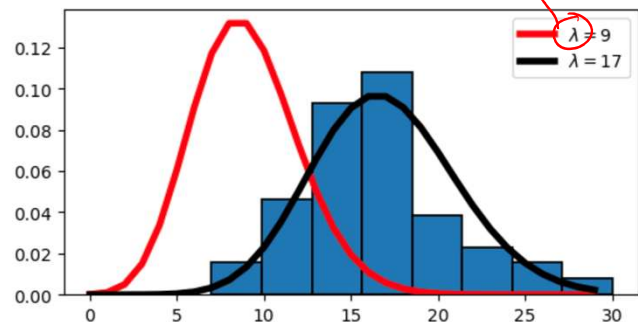
Fitting the data

- Since this is count data, we'll try to fit it using a Poisson distribution.
- From the data, we see that the most frequent value for the number of citations is ≈ 17 .

- Poisson distribution with a mean (λ) of 9 is not likely to have generated this dataset.

- Poisson distribution with a mean (λ) of 17 is more likely to have generated this dataset than a value of $\lambda = 9$.

- Why do the two distributions have different heights?

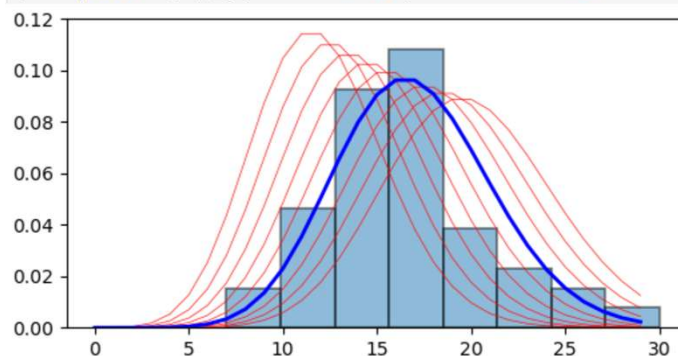


```
plt.figure(figsize=(6, 3))
plt.hist(citations_data["Counts"], bins=8, density=True,
        edgecolor='black', linewidth=1.2, alpha=0.5)

for i in range(12, 21):
    p_y = poisson.pmf(x, i)
    plt.plot(x, p_y, color='r', linewidth=0.5, alpha=.9)

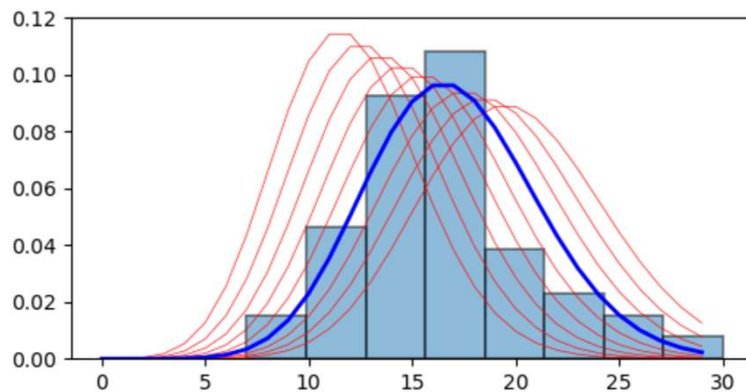
p_y = poisson.pmf(x, 17)
plt.plot(x, p_y, color='b', linewidth=2)
```

Handwritten red text: $\lambda \in [12, 20]$



Question

- The graph below shows clearly that some plots better fit the data.
- How could we formally (or informally) decide which one fits the best?



Computing the probability of observing the data

- We know how to compute the probability of an observation using the probability mass function (pmf).
 - For a Poisson distribution, the probability of a single observation is:
`poisson.pmf(x,  $\lambda$ )`
 - Since all observations are independent, the probability of the entire dataset is the product of the individual probabilities:
`np.prod(poisson.pmf(x,  $\lambda$ ))`
- We can evaluate this probability for different possible values of the parameter λ .
 - In practice, we restrict λ to a reasonable range of values.

An example

Compound probability of independent events:

- Given a fair coin, where $p(H) = p(T) = 1/2$
- If we flip the coin twice, the sample space is $\{HH, HT, TH, TT\}$ and probabilities of the different outcomes are:
 $p(HH) = p(H) \times p(H) = 1/4$, $p(HT) = p(H) \times p(T) = 1/4$
 $p(TH) = p(T) \times p(H) = 1/4$, $p(TT) = p(T) \times p(T) = 1/4$
- As expected, the probabilities sum to one.
- This multiplication rule works because the events are independent; i.e., obtaining a head on the first toss does not affect the outcome of the second flip.

Computing the likelihood of observing data

- If the events are independent, the probability of the data is the product of the probabilities.
- This is called the likelihood L of the data given the parameter λ .

$$\begin{aligned} L([x_1, x_2, \dots, x_{90}]|\lambda) \\ &= \text{pmf}(x_1, \lambda) \times \text{pmf}(x_2, \lambda) \times \dots \times \text{pmf}(x_{90}, \lambda) \\ &= \prod_{i=1}^{90} \text{pmf}(x_i, \lambda) \end{aligned}$$

```
data_point = 12
_lambda = 16
poisson.pmf(data_point, _lambda)
```

```
0.06612887239464967
```

```
data_point = 16
_lambda = 16
poisson.pmf(data_point, _lambda)
```

```
0.09921753162215541
```

```
poisson.pmf(citations_data["Counts"], _lambda)
```

```
array([0.00335386, 0.08300552, 0.00056393, 0.08138938, 0.05591951,
       0.04959665, 0.09921753, 0.06612887, 0.09338121, 0.06989938,
       0.09921753, 0.04959665, 0.09921753, 0.09921753, 0.09921753,
       0.06989938, 0.09301644, 0.08300552, 0.09338121, 0.0309857 ,
       0.08300552, 0.09921753, 0.02155527, 0.05591951, 0.0309857 ,
       0.09301644, 0.09338121, 0.09338121, 0.09921753, 0.01198747,
       0.04959665, 0.09301644, 0.00599374, 0.08300552, 0.09338121,
       0.09921753, 0.09921753, 0.08300552, 0.09301644, 0.09301644,
       0.09921753, 0.09338121, 0.08138938, 0.09921753, 0.08300552,
       0.09301644, 0.09338121, 0.06989938, 0.09921753, 0.06612887,
       0.09921753, 0.00105737, 0.09921753, 0.09921753, 0.08300552,
       0.09921753, 0.09301644, 0.06989938, 0.06612887, 0.02131106,
       0.05591951, 0.09301644, 0.09301644, 0.09301644, 0.02131106,
       0.09338121, 0.0309857 , 0.00919692, 0.06612887, 0.08138938,
       0.0340977 , 0.01437018, 0.09338121, 0.05591951, 0.0309857 ,
       0.09921753, 0.09301644, 0.04260534, 0.06612887, 0.04260534,
       0.09921753, 0.0340977 , 0.08138938, 0.06612887, 0.00919692,
       0.00919692, 0.09338121, 0.09921753, 0.09921753, 0.06612887])
```

90 probabilities.

$\sum \approx 1$

Question

- How can the likelihood of citation data be so low?

```
# Likelihood of the citation data
```

```
print(np.prod(poisson.pmf(citations_data["Counts"], _lambda)))
```

```
1.1152800100792418e-113
```

$\approx 1.16 \times 10^{-113}$

Understanding the likelihood

$$51 \times 51 \times 51$$

- Why is the resulting likelihood so small?
 - Think, for instance, about a sample from 3 days where the citations were (16, 15, 21).
 - The sample space is large and includes (0, 1, 8), (1, 10, 10), (21, 11, 28), ...
 - If the maximum number of citations daily is 50, then the sample space has a size $51^3 = 132,651$.
 - The probability of observing (16, 15, 21) is just one event (possibility) among many possible outcomes.
 - The sample space for 90 days (assuming 50 max citations per day) is $4.8 \times 10^{153} = 51^{90}$.
 - It therefore makes sense that the computed likelihood is so small.

```
_lambda = 16
print(np.prod(poisson.pmf((16, 15, 21), _lambda)))

0.0004194119981850487
```

```
print(np.prod(poisson.pmf((9, 11, 11), _lambda)))

5.24215504845479e-05
```

```
total_prob = 0
for i in range(0, 51):
    for j in range(0, 51):
        for k in range(0, 51):
            total_prob += np.prod(poisson.pmf((i, j, k), _lambda))

print(total_prob)

0.99999999999918858
```

Handwritten red annotations:
- "1st day" with an arrow pointing to the innermost loop (k).
- "day 2" with an arrow pointing to the middle loop (j).
- "day 3" with an arrow pointing to the outermost loop (i).

```

for i in range(12, 21):
    likelihood_data_i = np.prod(poisson.pmf(citations_data["Counts"], i))
    print(f"The likelihood of the data given  $\lambda = \{i\}$  is {likelihood_data_i:.5e}")

```

The likelihood of the data given $\lambda = 12$ is 7.22576e-142
 The likelihood of the data given $\lambda = 13$ is 1.30620e-129
 The likelihood of the data given $\lambda = 14$ is 3.68371e-121
 The likelihood of the data given $\lambda = 15$ is 5.43829e-116
 The likelihood of the data given $\lambda = 16$ is 1.11528e-113
 The likelihood of the data given $\lambda = 17$ is 7.05867e-114
 The likelihood of the data given $\lambda = 18$ is 2.67087e-116
 The likelihood of the data given $\lambda = 19$ is 1.05130e-120
 The likelihood of the data given $\lambda = 20$ is 6.87796e-127

Max. likelihood estimate

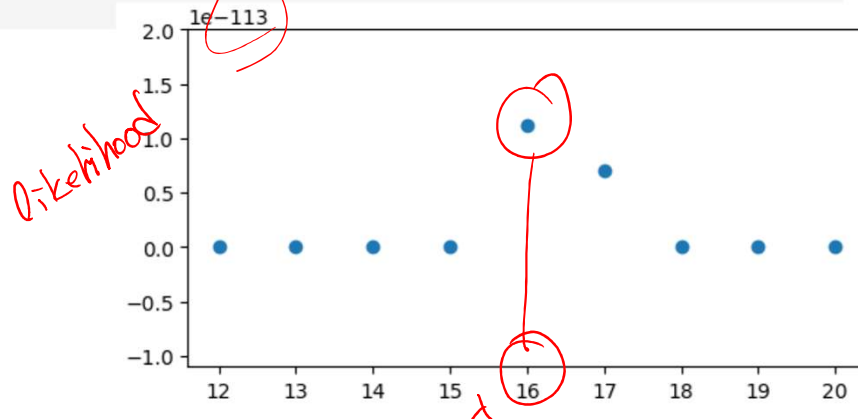
```

plt.figure(figsize=(6, 3))

x = range(12, 21)
l_x = [np.prod(poisson.pmf(citations_data["Counts"], i)) for i in x]

plt.scatter(x, l_x)
plt.ylim(-1.1e-113, 0.2e-112)
plt.show()

```



Understanding the likelihood (cont'd.)

- Working with small numbers can lead to likelihood to be very small.
 - Can lead to arithmetic floating-point underflow.
 - Results of a calculation smaller than what can actually be represented in memory.
- Rather than computing likelihood, we can calculate **log-likelihood**.
 - Scale changes but shape of the function does not.

Log-likelihood (cont'd.)

- We know that:

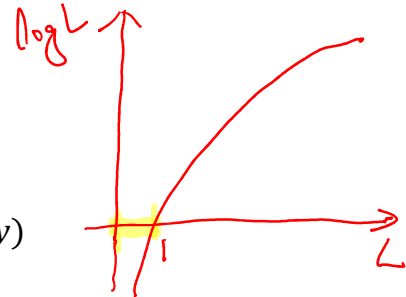
$$\log(x \cdot y) = \log(x) + \log(y)$$

- Therefore:

$$\begin{aligned} & \log(L([x_1, x_2, \dots, x_{90}]|\lambda)) \\ &= \log(\text{pmf}(x_1, \lambda) \times \text{pmf}(x_2, \lambda) \times \dots \times \text{pmf}(x_{90}, \lambda)) \\ &= \log(\text{pmf}(x_1, \lambda)) + \log(\text{pmf}(x_2, \lambda)) + \dots + \log(\text{pmf}(x_{90}, \lambda)) \\ &= \sum_{i=1}^{90} \log(\text{pmf}(x_i, \lambda)) \end{aligned}$$

Handwritten red note: $\in [0, 1]$

- The resulting values are no longer probabilities as they don't sum to 1.

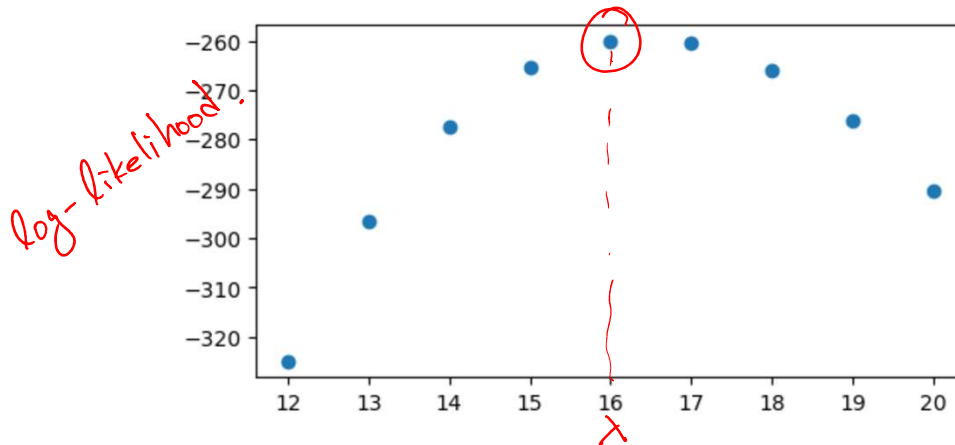


```

x = range(12, 21)
l_x = [np.sum(np.log(poisson.pmf(citations_data["Counts"], i))) for i in x]

plt.figure(figsize=(6, 3))
plt.scatter(x, l_x)
plt.show()

```



Maximum likelihood for the Poisson distribution

- We hypothesize that the data was generated from a Poisson distribution with unknown parameter λ .
 - Among all possible λ values, the data is most likely under one of those.
 - We say that this value maximizes the likelihood.
- We can quickly show that the value that maximized the likelihood merely is the sample mean!


```

      >>> np.mean(citations_data["Counts"])
      16.41
      
```
- While we assumed that the data is best represented using a Poisson, we could have manually tested other distributions and selected the one that provides the highest likelihood.
 - Therefore, this approach is not only suitable for estimating parameters but also for making decisions.

Example: A/B testing

- In an A/B test, we are interested in comparing two versions of a webpage.
 - We would like to determine which one performs better at a task X.
- E.g.: Does Version A (new) or Version B (old) perform better at user sign-ups?



Maximum likelihood for a Binomial distribution

- For now, we are only concerned with version A.
- Suppose we test 8 individuals (unrealistically small sample for illustration) using version A and observe 5 sign-ups.
- What can we infer about:
 - The type of data we are observing?
 - The parameter that gave rise to this outcome?

(n, p_A)

data

$5/8 \approx 0.625$

Maximum likelihood for a Binomial distribution (cont'd.)

1. The data are distributed according to a binomial distribution:
 - $n = 8$ and $p \in [0, 1]$
2. Intuitively, we know that the parameter cannot be too small or too big.
 - From the data, p is somewhere in the upper side of the range.
 - For example, the data are more likely with a $p = 0.8$ than with $p = 0.03$.
- We can determine using the same approach used with traffic citations data.

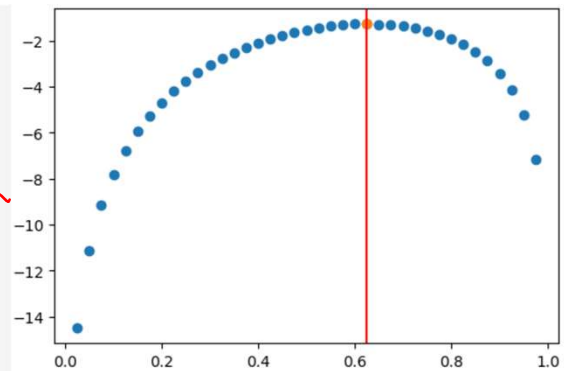
```
x = np.arange(0.025, 1, 0.025)
llh_x_vals = []

for p in x:
    # data size = 1. No need to sum in this example.
    llh_x_i = np.sum(np.log(binom.pmf(5, n, p)))
    llh_x_vals.append(llh_x_i)

mle_p = x[np.argmax(llh_x_vals)]
print(f"MLE of p = {mle_p}")

plt.figure(figsize=(6, 4))
plt.scatter(x, llh_x_vals)
plt.scatter(x[np.argmax(llh_x_vals)], np.max(llh_x_vals))
plt.axvline(mle_p, color='r')
plt.show()
```

MLE of p = 0.625



Maximum likelihood for a binomial distribution (cont'd.)

- If we toss a coin 100 times and observe 85 heads, then we assume the probability of heads is 0.85.
- Since 5 out of 8 clients signed up, we assume that $p = 5/8 = 0.625$.
 - Therefore, the maximum likelihood result ($p = 0.625$) is consistent with our intuition $p = 5/8 = 0.625$.

Maximum likelihood estimation

- The MLEs for parameters of common distributions can be computed analytically.
 - For example, the maximum estimator of the mean in a Gaussian is simply the sample mean, and the estimator of the standard deviation is the sample's standard deviation.

$$\begin{aligned} \text{mean} \quad \hat{\mu} &= \frac{1}{n} \sum_{i=1}^n x_i \\ \text{std} \quad \hat{\sigma} &= \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu})^2} \end{aligned}$$

Maximum likelihood estimation (cont'd.)

- Recall that MLEs are simply **point-estimates** that maximize the likelihood of data.
- Do not give us a probability of the value we estimate.
 - We do not have the probability or confidence that it is the correct parameter of the population.
 - They don't even provide confidence intervals, as the bootstrap approach did.
- The questions is then: **"How different could this estimate have been if we had observed different sample?"**